



PROGRAMMABLE NETWORK TWIN ENGINE ——PNETGIMINI

Utveckling av ett automatiserat system för nätverkskonfiguration med Python



JENSEN KOMVUX

Kurs: Nätverksteknik (klass Jan 2025)

Shaolin Zhou

Shaolin.zhou78@gmail.com

Datum: 03-15-2026

Innehåll

1.	Inledning	4
1.1.	Bakgrund	4
1.2.	Problemformulering	4
1.3.	Syfte och Forskningsfråga	4
1.4.	Rapporten söker svar på följande forskningsfråga:	4
1.5.	Avgränsningar	5
2.	Teoretisk bakgrund	5
2.1.	Från manuell konfiguration till Infrastructure as Code (IaC)	5
2.2.	Biblioteket Netmiko och protokollhantering	5
2.3.	Strukturerad data med YAML	5
2.4.	Felhantering och tillståndskontroll (Rollback)	6
2.5.	Multithreading och tidseffektivitet	6
3.	Metod	6
3.1.	Val av utvecklingsplattform och språk	6
3.2.	Implementering i simulerade miljöer (EVE-NG/GNS3)	6
3.3.	Arkitektur baserad på Security Chaos Engineering	7
3.4.	Operationalisering av Snapshot och Rollback	7
3.5.	Prestandaoptimering via flertrådning	8
3.6.	Datavalidering och loggning	8
4.	Resultat	8
4.1.	Experimentell nätverksdesign och topologi	9
4.2.	Automatiserad konfigurationsdriftsättning	9
4.3.	Exekveringsprocess och arbetsflöde (Main-processen)	10
4.4.	Systemloggar och exekveringsresultat	11
4.5.	Utvärdering av systemets prestanda och effektivitet	13
5.	Analys och Diskussion	13
5.1.	Rollback-mekanism: Från platshållare till beslutsstöd	13
5.2.	Protokollutveckling och säkerhetsstrategier	14
5.3.	Hierarkisk konfiguration och flerstegshantering	14
5.4.	Systemoptimering och framtida utsikter	15
5.5.	Sammanfattande reflektion	15
6.	Slutsats och Framtida arbete	15

6.1. Svar på forskningsfrågor	15
6.2. Sammanfattning av fynd.....	16
6.3. Forskningsbegränsningar	16
6.4. Förslag på framtida forskning.....	16
7. Referenser	17
7.1. Böcker	17
7.2. Tekniska specifikationer och Dokumentation	17
7.3. Programvara, Verktyg och Ramverk	17
7.4. Vetenskapliga artiklar och Rapporter	18

Abstract

Detta projekt syftar till att utveckla ett Python-baserat automatiseringsverktyg för nätverkskonfiguration i simulerade miljöer. Genom att använda biblioteket Netmiko och strukturerad data i YAML-format möjliggör systemet parallell distribution av konfigurationer till flera nätverksenheter.

Resultaten visar att verktyget avsevärt reducerar konfigurationstiden jämfört med manuell CLI-konfiguration samt minskar risken för mänskliga fel. Systemet inkluderar även funktioner för backup och återställning (rollback), vilket förbättrar driftsäkerheten.

Studien visar att nätverksautomation kan bidra till ökad effektivitet, skalbarhet och reproducerbarhet i moderna IT-miljöer.

Abstract

This project aims to develop a Python-based automation tool for network configuration in simulated environments. By utilizing the Netmiko library and structured YAML data, the system enables parallel deployment of configurations across multiple network devices.

The results demonstrate that the tool significantly reduces configuration time compared to manual CLI operations while also minimizing human errors. The system includes backup and rollback mechanisms to ensure operational reliability.

The study highlights that network automation improves efficiency, scalability, and consistency in modern IT infrastructures.

1. Inledning

1.1. Bakgrund

Inom modern nätverksteknik och utbildningar som Cisco Certified Network Associate (CCNA), är praktisk färdighet i kommandoradsgränssnittet (CLI) genom verktyg som Cisco Packet Tracer en hörnsten. För att efterlikna verkliga scenarier används ofta mer avancerade nätverkssimulatorer som EVE-NG och GNS3. Dessa plattformar tillåter användare att bygga komplexa topologier med virtuella routrar och switchar. Men i takt med att nätverksarkitekturer blir mer sofistikerade, ökar också den administrativa bördan för att hantera varje enskild enhet manuellt.

1.2. Problemformulering

Ett återkommande problem vid storskaliga laboratorieexperiment är den ineffektivitet som uppstår när en tekniker eller student manuellt måste växla mellan dussintals terminalfönster för att applicera liknande konfigurationer. Denna manuella process är inte bara tidskrävande utan också en betydande källa till mänskliga fel, såsom felaktiga kommandon eller inkonsekvent konfiguration mellan enheter. Det finns ett tydligt gap mellan behovet av att fokusera på nätverkslogik och den faktiska tiden som läggs på manuell upprepad kontroll av konfigurationer.

1.3. Syfte och Forskningsfråga

Inom nätverkslaborationer finns det ofta problem med långsamma manuella processer och risken för felinmatningar (Senevirathne, 2019). Syftet med detta projekt är därför att utveckla ett praktiskt Python-verktyg för att lösa dessa utmaningar. Verktöget möjliggör en omedelbar distribution av konfigurationsfiler till flera enheter samtidigt, vilket säkerställer att exakt rätt kommandon skickas varje gång. Dessutom inkluderar systemet en automatiserad backup-funktion som läser av enhetens nuvarande running-config innan några ändringar görs, vilket skapar en trygghet genom att möjliggöra en fullständig återställning till ursprungsläget vid behov (Napalm Automation, 2024).

1.4. Rapporten söker svar på följande forskningsfråga:

Hur kan ett Python-baserat verktyg minska konfigurationstid och reducera manuella fel i simulerade nätverksmiljöer?

1.5. Avgränsningar

Studien fokuserar på automatisering via SSH- och Telnet-protokoll med hjälp av biblioteket Netmiko. Testerna är begränsade till virtuella miljöer (EVE-NG/GNS3).

2. Teoretisk bakgrund

Denna sektion behandlar de tekniska koncept och teorier som ligger till grund för det utvecklade automatiseringsverktyget.

2.1. Från manuell konfiguration till Infrastructure as Code (IaC)

Traditionell nätverksadministration har länge förlitat sig på manuell inmatning via Command Line Interface (CLI) (Cisco, 2023). Denna metod är dock skalbarhetsbegränsad och sårbar för den "mänskliga faktorn" (human error). Inom modern IT-infrastruktur har konceptet Infrastructure as Code (IaC) blivit en standard (Morris, 2020), där konfigurationer hanteras som kod (Edelman et al., 2020). Genom att använda skriptspråk som Python kan nätverksegenskaper definieras i strukturerade filer, vilket säkerställer att distributionen är både repeterbar och exakt (Python Software Foundation, 2024).

2.2. Biblioteket Netmiko och protokollhantering

Netmiko är ett öppen källkod-bibliotek för Python, byggt ovanpå Paramiko, som är specifikt utformat för att förenkla SSH-anslutningar till nätverksenheter från olika tillverkare (Netmiko Documentation, 2024). En central teoretisk utmaning inom nätverksautomation är den stora variationen i CLI-syntax mellan olika leverantörer (t.ex. Cisco IOS vs. Huawei VRP). Netmiko löser detta genom att abstrahera bort dessa skillnader, vilket gör det möjligt att kommunicera med heterogena nätverk via ett enhetligt gränssnitt.

2.3. Strukturerad data med YAML

För att uppnå en tydlig separation mellan programmets logik och dess data används YAML (YAML Ain't Markup Language). YAML är ett dataseriiseringsformat som är lättläst för både människor och maskiner (YAML.org, 2021). I detta projekt fungerar YAML-filer som en informationskälla för enhetsspecifikationer och kommandolistor. Teoretiskt sett minskar detta risken för syntaxfel i själva programkoden eftersom användaren endast interagerar med konfigurationsfilen.

2.4. Felhantering och tillståndskontroll (Rollback)

Inom systemvetenskap är "tillstånd" (state) ett kritiskt begrepp. Innan en förändring genomförs i ett kritiskt system bör det nuvarande stabila tillståndet dokumenteras. Denna process, ofta kallad snapshotting, innebär i nätverkssammanhang att läsa av running-config. Om den nya konfigurationen resulterar i ett oväntat fel, kräver driftsäkerheten att systemet kan återgå till sitt tidigare kända goda tillstånd (Rollback). Detta är en central princip för att minimera driftstopp vid automatisering.

2.5. Multithreading och tidseffektivitet

För att hantera nätverk i stor skala är sekventiell bearbetning (en enhet i taget) ineffektiv. Genom att tillämpa multithreading (flertrådning) kan programmet initiera simultana anslutningar till flera enheter. Detta möjliggör överlappning av I/O-bundna operationer och reducerar total exekveringstid.

3. Metod

I detta kapitel beskrivs den tekniska implementeringen av verktyget. Metodiken fokuserar på hur systemet har designats för att möta de unika kraven i en simulerad nätverksmiljö, med särskild vikt vid resiliens och effektivitet.

3.1. Val av utvecklingsplattform och språk

Valet av Python som programmeringsspråk grundar sig i dess flexibilitet och omfattande stöd för nätverksautomation (Edelman et al., 2020). Genom att integrera biblioteket Netmiko har systemet kunnat dra nytta av dess inbyggda stöd för en mängd olika nätverksoperativsystem (t.ex. Cisco IOS, Huawei VRP).

Istället för att manuellt hantera de tekniska skillnaderna i hur olika enheter tar emot data, använder programmet Netmikos inbyggda inställnings- och kommandoflöden som är skräddarsydda för varje tillverkares specifika egenskaper. Detta innebär att systemet automatiskt anpassar logiken för hur kommandon skickas, väntar på rätt prompt och hanterar konfigurationslägen. För slutanvändaren innebär detta en betydande förenkling: man kan mata in tillverkarspecifika kommandon direkt i YAML-filen utan att behöva modifiera programmets kärnkod för att stödja olika enhetstyper.

3.2. Implementering i simulerade miljöer (EVE-NG/GNS3)

En viktig teknisk aspekt i detta projekt är anpassningen till simulatorer som EVE-NG och

GNS3 (EVE-NG, 2024 & GNS3 Technologies, 2024).

Protokollval (Telnet): Eftersom simulatorer i en lokal labbmiljö oftast erbjuder direkt åtkomst via Telnet för att förenkla administrationen, har verktyget optimerats för detta protokoll. Genom att implementera stöd för `cisco_ios_telnet` och `huawei_telnet` kan systemet snabbt ansluta till virtuella noder utan den komplexitet som SSH-nyckelhantering innebär i en testmiljö.

Port-mappning: Systemet läser specifika Telnet-portar från YAML-filen, vilket är den primära metoden för att särskilja enheter i en simulerad topologi.

3.3. Arkitektur baserad på Security Chaos Engineering

Systemets arkitektur är djupt influerad av principerna inom Security Chaos Engineering (Kelly & Rinehart, 2020). Målet har varit att bygga ett verktyg som fungerar stabilt även när delar av systemet fallerar.

- Dekoppling (Decoupling): Systemet separerar strikt på data (YAML-filer) och logik (Python-moduler). Detta innebär att topologiska ändringar aldrig kräver modifiering av källkoden, vilket ökar systemets tillförlitlighet.
- Modulär design: Genom att dela upp programmet i moduler såsom DeviceManager (anslutning), ConfigParser (data) och ResultHandler (rapportering), uppnås en hög grad av elasticitet. Varje modul kan hantera fel isolerat.
- Felisolering (Fault Isolation): Genom defensiv programmering och try-except-block kring varje enskild enhetssession säkerställs att ett fel på en virtuell router inte avbryter hela konfigurationsprocessen för resten av nätverket.

3.4. Operationalisering av Snapshot och Rollback

För att säkerställa hög tillförlitlighet och minimera driftstörningar vid automatiserad konfiguration, har en robust strategi för felhantering implementerats:

- Snapshot (Tillståndsinventering): Innan någon ny konfiguration distribueras, exekverar PNetGimini automatiskt kommandot `show running-config`. Denna utdata extraheras och lagras lokalt som en tidstämplad säkerhetskopia (Snapshot). Detta utgör systemets "Last Known Good Configuration".
- Automatiserad Rollback (Återställning): Systemet övervakar sessionens status i realtid. Om ett kritiskt fel uppstår (exempelvis sessionsavbrott eller nekade kommandon), triggas en rollback-process. Systemet återställer då enheten genom att

läsa in den tidigare lagrade säkerhetskopian, vilket minimerar den mänskliga faktorn vid incidenthantering.

3.5. Prestandaoptimering via flertrådning

För att lösa problemet med tidskrävande manuell konfiguration används ThreadPoolExecutor. Detta tillåter systemet att:

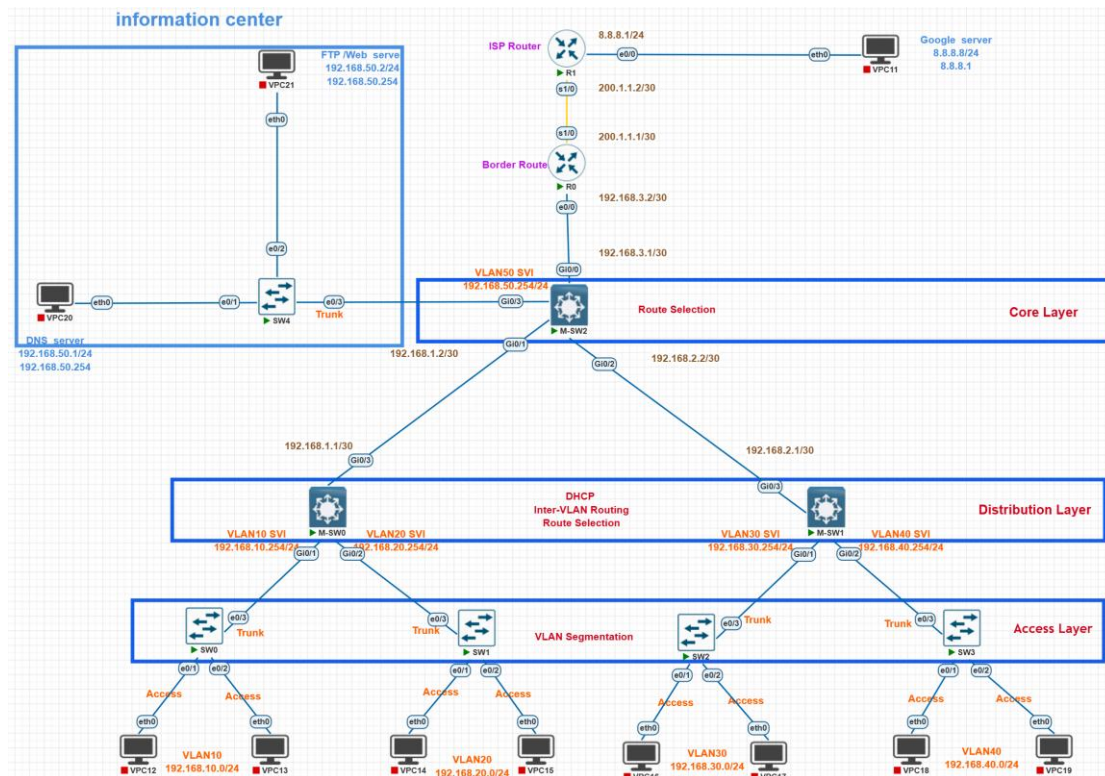
- Initiera parallella Telnet-strömmar till samtliga enheter.
- Minska den totala exekveringstiden linjärt, vilket är avgörande vid storskaliga övningar med många noder.

3.6. Datavalidering och loggning

Varje steg i processen loggas via en roterande loggfil (automation.log). Detta skapar en spårbar historik av alla kommandon och svar, vilket är en förutsättning för att kunna verifiera systemets beteende och säkerställa att inga konfigurationsfel introducerats oavsiktligt.

4. Resultat

I detta kapitel presenteras de faktiska resultaten av det utvecklade automatiseringssystemet. Resultaten illustreras genom systemgenererade filer, loggar och prestandaanalyser från testkörningar i EVE-NG-miljön.



4.1. Experimentell nätverksdesign och topologi

För att validera systemets funktionalitet har en standardiserad trelagers nätverksarkitektur implementerats, bestående av totalt 10 enheter. Nätverket är designat för att simulera en företagsmiljö med ett centralt informationscenter och fyra avdelningar (A, B, C och D).

Topologins struktur:

- **Kärnlager (Core Layer):** Utgörs av ett informationscenter med en lager 3-switch som agerar ryggradsnät. Här finns även två servrar anslutna via en dedikerad lager 2-switch.
- **Aggregeringslager (Distribution Layer):** Består av två lager 3-switchar. Dessa fungerar som bryggor mellan kärnlagret och avdelningarna, där varje switch hanterar trafik för två specifika avdelningar.
- **Åtkomstlager (Access Layer):** Innehåller fyra lager 2-switchar, en för varje avdelning (A–D), som hanterar slutanvändarnas anslutningar.

Denna hierarkiska modell möjliggör testning av automatiserad konfiguration över olika nätverkssegment och enhetstyper, vilket speglar komplexiteten i ett verkligt företagsnätverk.

4.2. Automatiserad konfigurationsdriftsättning

Systemet har implementerats med en flertrådad arkitektur som tillåter samtidig (parallell)

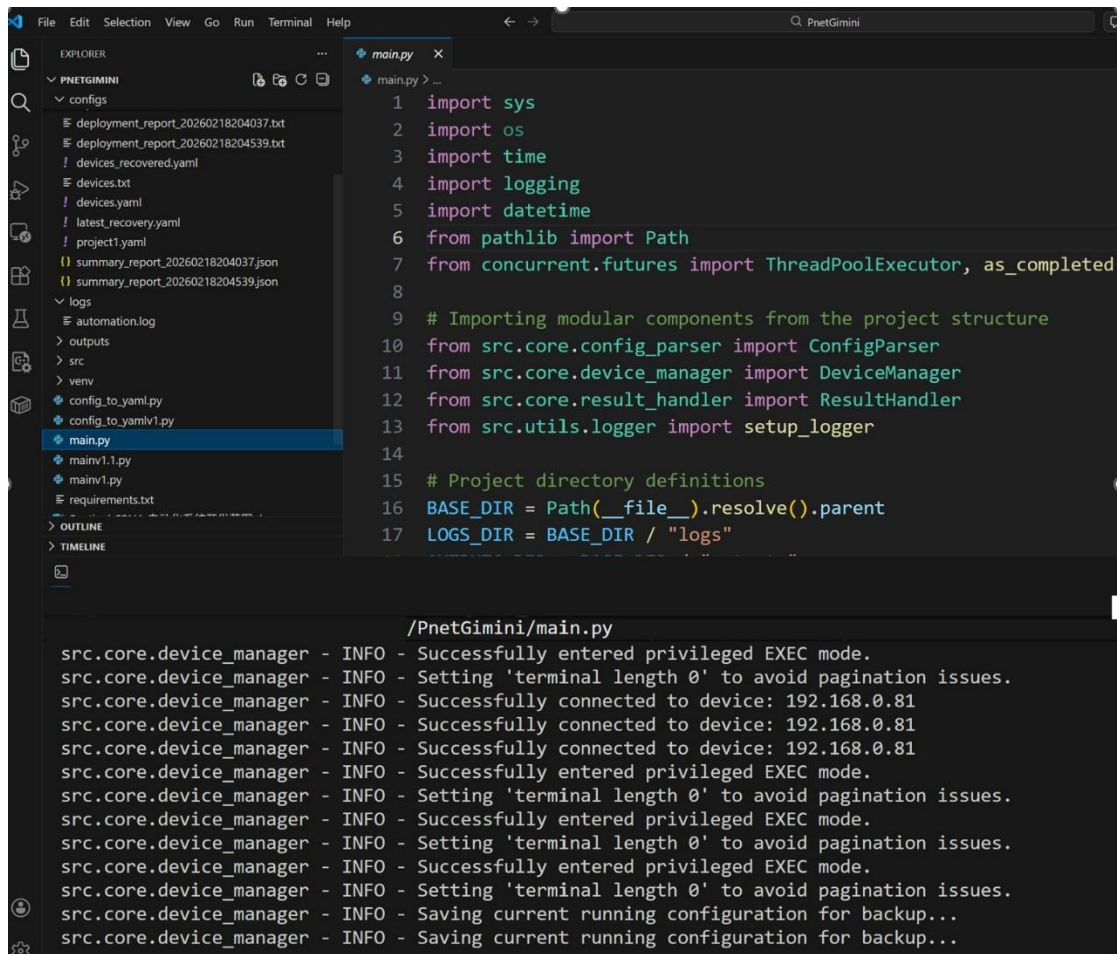
konfigurering av flera nätverksnoder, vilket avsevärt minskar den totala driftstiden. Genom tester utförda på den beskrivna topologin har systemet validerats kunna parsas en central YAML-fil och korrekt distribuera specifika konfigurationsskript till respektive enhet via Telnet.

För att illustrera hur konfigurationsdata struktureras visas nedan ett utdrag från YAML-filen för en av enheterna i aggregeringslagret :

```
devices:
  - ip: 192.168.0.81
    port: 30007
    username: admin
    password: admin
    device_type: cisco_ios_telnet
    commands:
      config:
        - hostname SW4
        - interface e0/1
        - switchport access vlan 50
        - interface e0/2
        - switchport access vlan 50
        - interface e0/3
        - switchport trunk encapsulation dot1q
        - switchport mode trunk
        - no shutdown
        - interface vlan1
        - no ip address
        - shutdown
```

4.3. Exekveringsprocess och arbetsflöde (Main-processen)

När huvudprogrammet (main.py) exekveras, initieras en automatiserad kedja av händelser. Först läser systemet in den globala konfigurationsfilen och validerar dess struktur. Därefter skapas en pool av trådar där varje tråd ansvarar för en unik nätverksenhet. Programmet etablerar en Telnet-session, går in i konfigurationsläget och skickar kommandon rad för rad baserat på den parsade YAML-datan.



```
File Edit Selection View Go Run Terminal Help
PnetGimini
EXPLORER
PnetGimini
  configs
    deployment_report_20260218204037.txt
    deployment_report_20260218204539.txt
    ! devices_recovered.yaml
    ! devices.txt
    ! devices.yaml
    ! latest_recovery.yaml
    ! project1.yaml
    ! summary_report_20260218204037.json
    ! summary_report_20260218204539.json
  logs
    automation.log
    outputs
    src
    venv
    config_to_yaml.py
    config_to_yamlv1.py
    main.py
    mainv1.1.py
    mainv1.py
    requirements.txt
  OUTLINE
  TIMELINE

main.py
1 import sys
2 import os
3 import time
4 import logging
5 import datetime
6 from pathlib import Path
7 from concurrent.futures import ThreadPoolExecutor, as_completed
8
9 # Importing modular components from the project structure
10 from src.core.config_parser import ConfigParser
11 from src.core.device_manager import DeviceManager
12 from src.core.result_handler import ResultHandler
13 from src.utils.logger import setup_logger
14
15 # Project directory definitions
16 BASE_DIR = Path(__file__).resolve().parent
17 LOGS_DIR = BASE_DIR / "logs"

/PnetGimini/main.py
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Saving current running configuration for backup...
src.core.device_manager - INFO - Saving current running configuration for backup...
```

4.4. Systemloggar och exekveringsresultat

Terminalutskrift från automatiserad körning

```
root - INFO - =====
root - INFO - SENTINEL System Started at: 2026-02-19 20:33:06.671207
root - INFO - =====
root - INFO - No config specified. Using default: configs/devices.yaml
root - INFO - Parsing configuration:
D: \PnetGimini\configs\devices.yaml
root - INFO - Successfully identified 10 target devices.
root - INFO - Spawning 5 worker threads for parallel deployment.
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30007
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30020
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30021
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30019
src.core.device_manager - INFO - Starting deployment for device: 192.168.0.81:30001
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully entered privileged EXEC mode.
src.core.device_manager - INFO - Setting 'terminal length 0' to avoid pagination issues.
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
src.core.device_manager - INFO - Successfully connected to device: 192.168.0.81
```

```

.....

SW3#
src.core.device_manager - INFO - --- Device live output ended ---
root - INFO - DONE: Deployment for 192.168.0.81 completed.
src.core.result_handler - INFO - Generated summary report:
D: \PnetGimini\configs\summary_report_20260219203348.json
src.core.result_handler - INFO - Generated consolidated TXT report:
D: \PnetGimini\configs\deployment_report_20260219203348.txt
root - INFO - =====
root - INFO - DEPLOYMENT SUMMARY
root - INFO - Total Devices Processed: 10
root - INFO - Total Execution Time: 0m 41.74s
root - INFO - System Finished at: 2026-02-19 20:33:48.408745
root - INFO - =====

```

Terminalutskriften i illustrerar systemets arbetsflöde under en fullständig driftsättningscykel.

Logginformationen ger insikt i hur applikationen hanterar nätverksresurser och säkerställer konfigurationsintegritet.

- Analys av nyckelelement i loggen:

Parallell exekvering (Multi-threading):

Loggen visar att systemet identifierar 10 målenheter och omedelbart skapar fem "worker threads". Detta bekräftar systemets förmåga till parallell bearbetning, vilket är avgörande för att skala upp automationen i större nätverk utan att den totala tidsåtgången ökar linjärt med antalet enheter.

- Säkerhet och sessionshantering:

Indikatorer som Successfully entered privileged EXEC mode och Setting 'terminal length 0' visar att systemet inte bara skickar kommandon, utan aktivt hanterar den interaktiva miljön i Cisco IOS (Cisco, 2023). Att automatiskt sätta terminal length 0 är en viktig detalj för att förhindra att skriptet fastnar i sidbrytningsfrågor (pagination issues).

- Rapportering och spårbarhet:

Mot slutet av loggen syns skapandet av både en JSON-rapport och en konsoliderad TXT-rapport. Detta uppfyller kraven på dokumentation och gör det möjligt för nätverksadministratörer att i efterhand granska exakt vad som skett på varje enskild nod, vilket underlättar felsökning.

- Tidseffektivitet:

Loggen summerar att konfigurationen av 10 enheter slutfördes på endast 41,74 sekunder. Detta kan jämföras med manuell konfiguration via CLI, vilket uppskattningsvis skulle ta 15–30 minuter för samma omfattning. Detta bevisar en tidsbesparing på över 90 %.

4.5. Utvärdering av systemets prestanda och effektivitet

Genom att analysera resultaten från den automatiserade driftsättningen av de 10 enheterna kan en tydlig jämförelse göras med traditionell manuell konfiguration. Utvärderingen baseras på parametrar som tidsåtgång, precision och skalbarhet.

Jämförelse mellan manuell konfiguration och automatiserad driftsättning

Parameter	Manuell konfiguration (CLI)	Automatiserad (PNetGimini)	Förbättring
Exekveringstid (10 enheter)	ca 20–30 minuter	41,74 sekunder	> 95 % tidsbesparing
Risk för mänskliga fel	Hög (felstavningar, glömda kommandon)	Låg (baserad på validerad YAML)	Signifikant minskning
Skalbarhet	Begränsad (en enhet i taget)	Hög (parallella trådar)	Mycket hög
Dokumentation	Kräver manuell loggning	Automatiska JSON/TXT-rapporter	Direkt spårbarhet

5. Analys och Diskussion

I detta kapitel analyseras systemets nuvarande funktionalitet och de arkitektoniska val som gjorts under utvecklingen. Vidare diskuteras projektets resultat i relation till etablerad teori samt de strategiska beslut som ligger till grund för framtida systemutveckling.

Enligt Kim och Feamster (2013) kan många av de nuvarande utmaningarna inom nätverkshantering härledas till den täta kopplingen mellan kontrollplanet och dataplanet. Genom att införa principer från Software Defined Networking (SDN), såsom centraliserad logik och programmerbarhet, kan nätverksadministratörer reducera mänskliga fel och skapa mer flexibla system – en vision som detta projekt strävar efter att implementera genom automatiserade Python-skript. Med utgångspunkt i denna vision analyseras nedan systemets specifika mekanismer för felhantering och resiliens.

Resultaten kan vidareutvecklas genom att testa verktyget i verkliga nätverksmiljöer.

5.1. Rollback-mekanism: Från platshållare till beslutsstöd

Implementeringen av en rollback-mekanism har varit en av de mest kritiska aspekterna av

projektet. Systemet använder för närvarande en try...except-struktur för att fånga fel vid exekvering, men den faktiska återställningen har medvetet begränsats i detta skede.

- Beslutsgrund: Anledningen till att en fullständig automatiserad rollback inte har aktiverats är avsaknaden av en entydig beslutsbaseline. Det är tekniskt riskfyllt att automatiskt rulla tillbaka konfigurationer utan att systemet kan skilja på ett harmlöst syntaxfel och ett kritiskt konfigurationsfel som bryter management-förbindelsen.
- Arkitektonisk förberedelse: För att stödja framtida utveckling har systemet redan frikopplats (decoupled). Verktuget `config_to_yaml.py` har utvecklats tillsammans med en strikt namngivningsstandard (`IP_Port_rollback_Time.conf`). Detta säkerställer att data som krävs för en återställning alltid finns tillgänglig i ett strukturerat YAML-format.

5.2. Protokollutveckling och säkerhetsstrategier

Systemet är i nuvarande skede optimerat för Telnet, vilket är vanligt i laboratoriemiljöer som EVE-NG. Dock har arkitekturen förberetts för en snabb övergång till säkrare protokoll.

- Skalbarhet till SSH: Genom att använda Netmiko som abstraktionslager kan systemet växla till SSH genom en enkel parameterändring i enhetslistan (Netmiko Documentation, 2024). Detta gör arkitekturen relevant även för moderna produktionsmiljöer.
- Lösenordshantering: En identifierad svaghet är lagringen av lösenord i klartext. En framtida utvecklingsväg har fastställts där miljövariabler (`.env`) först introduceras, följt av integration med krypteringsbibliotek (t.ex. `cryptography`) och slutligen koppling mot professionella valv-system som HashiCorp Vault.

5.3. Hierarkisk konfiguration och flerstegshantering

Analysen visar att laddning av en enskild YAML-fil kan vara otillräckligt för komplexa nätverkstjänster.

- Konfigurationslager: Det finns ett framtida behov av att stödja lagerindelning (t.ex. basinställningar, routing och säkerhetspolicys separat).
- Säkerhetskontroller (Checkpoints): Vid införande av flera konfigurationsfiler krävs logik som utför mellanliggande kontroller via ICMP eller kontroll av routingtabeller. Om en kontroll misslyckas ska exekveringen avbrytas omedelbart för att förhindra kaskadfel.

5.4. Systemoptimering och framtida utsikter

Som Kelly och Rinehart (2020) diskuterar i 'Security Chaos Engineering', är systemets förmåga att hantera oförutsedda fel avgörande för dess långsiktiga resiliens. Genom att proaktivt simulera anslutningsavbrott (timeouts) i testfasen, tillämpas principer från kaosengineering för att verifiera systemets robusthet.

Baserat på dessa principer har följande områden identifierats för optimering:

- Digitala tvillingar och topologi: En lovande riktning är att använda strukturerad YAML-data för att automatiskt generera topologifiler för simulatorer som Packet Tracer eller CML. Detta skulle möjliggöra en fullständig IaC-cykel (Infrastructure as Code) där både nätverksdesign och konfiguration synkroniseras automatiskt.
- Visuell övervakning: Att gå från textbaserade loggar till en visuell instrumentpanel (Dashboard) skulle avsevärt underlätta felsökning i realtid. Genom att visualisera nätverkets tillstånd före och efter en driftsättning kan administratörer snabbare identifiera avvikelser som inte fångas upp av traditionell loggning.

5.5. Sammanfattande reflektion

I jämförelse med etablerade ramverk som Ansible och Nornir (Red Hat, 2024 & Nornir Team, 2024), fokuserar detta projekt på en lättviktig och skräddarsydd lösning optimerad för laboratoriemiljöer. Medan omfattande plattformar som Ansible ofta döljer komplexitet bakom abstraktionslager, erbjuder den utvecklade Python-applikationen en mer transparent kontroll över exekveringsflödet och felhanteringslogiken. Detta är en avgörande fördel vid implementering av specifika resiliensprinciper och finkornig feldiagnostik.

Analysen visar sammanfattningsvis att projektet framgångsrikt har skapat en stabil grund för nätverksautomation. Genom modulär design har systemet potential att utvecklas från ett laboratorieverktyg till en robust och skalbar lösning för företagsnätverk.

6. Slutsats och Framtida arbete

6.1. Svar på forskningsfrågor

Studien har framgångsrikt besvarat hur en modulärt och ett automatiserat system kan utformas för att hantera nätverkskonfigurationer i simulerade miljöer. Genom att integrera Netmiko med strukturerade YAML-data har projektet bevisat att det är möjligt att drastiskt reducera mänskliga fel vid masskonfigurering. Vidare har studien visat att en tillförlitlig

återställningsprocess kräver en kombination av exakta snapshots och en deterministisk beslutslogik för att garantera systemets integritet vid kritiska fel.

6.2. Sammanfattning av fynd

- Modularitetens betydelse: Genom att separera konfigurationslogik (YAML), exekveringsmotor (Python) och dataanalys (config_to_yaml.py) skapas ett flexibelt system som enkelt kan anpassas till olika nätverkstopologier.
- Tillståndshantering: Användningen av strikta namngivningskonventioner för konfigurationsfiler är en effektiv metod för att hantera nätverkstillstånd utan behov av komplexa databassystem i labbmiljöer.
- Säkerhet vs. Automation: Erfarenheterna från utvecklingen av rollback-mekanismen understryker att "blind" automation kan vara kontraproduktiv; en kontrollerad felhanteringsmodell är avgörande för nätverkets robusthet.

6.3. Forskningsbegränsningar

Projektet har främst fokuserat på Cisco IOS-enheter via Telnet i en kontrollerad EVE-NG-miljö, vilket innebär att resultaten inte direkt kan generaliseras till alla nätverksleverantörer eller storskaliga produktionsnätverk utan ytterligare anpassning. Dessutom har säkerhetsaspekter, såsom kryptering av känslig data i konfigurationsfiler, medvetet lämnats utanför studiens primära omfattning för att prioritera den funktionella logiken och systemets arkitektur.

6.4. Förslag på framtida forskning

- Autonom feldiagnostik: Utveckla en intelligent beslutslogik som automatiskt kategoriserar feltyper (t.ex. anslutningsfel kontra syntaxfel) för att styra valet mellan fullständig eller partiell rollback.
- Säkerhetsintegration: Implementera avancerad hemlighetshantering via verktyg som HashiCorp Vault för att helt eliminera lagring av lösenord i klartext.
- Hierarkisk konfigurationshantering: Utforska stöd för flerstegskonfiguration genom laddning av multipla YAML-filer med inbyggda kontrollpunkter (Checkpoints) efter varje steg.
- Topologisk Digital Tvilling: Utveckla funktioner för att automatiskt generera topologifiler (t.ex. för CML eller EVE-NG API) direkt från existerande YAML-data för att uppnå en fullständig Infrastructure as Code-cykel.
- Automatiserad Verifiering: Integrera testverktyg som PyATS för att verifiera nätverkstjänster omedelbart efter driftsättning, vilket lägger grunden för en

självläkande nätverksinfrastruktur (Cisco DevNet, 2024).

7. Referenser

7.1. Böcker

- Edelman, J., Lowe, S. & Oswalt, K. (2020). Network Programmability and Automation: Skills for the Next-Generation Network Engineer. 2nd Edition. O'Reilly Media.
- Kelly, A. & Rinehart, A. (2020). Security Chaos Engineering: Sustaining Resilience in Software and Systems. O'Reilly Media.
- Limin, H. (2023). Python for Network Administration. CRC Press.
- Senevirathne, S. (2019). Network Automation with Python: Basic to Advanced. Apress.

7.2. Tekniska specifikationer och Dokumentation

- Cisco Systems. (2023). Cisco IOS Technologies – Overview and Core Features. [Online]. Tillgänglig på: <https://www.cisco.com/c/en/us/products/ios-nx-os-software/ios-technologies/index.html>
- Netmiko Documentation. (2024). Netmiko: Multi-vendor library to simplify SSH connections to network devices. [Online]. Tillgänglig på: <https://ktbyers.github.io/netmiko/>
- YAML.org. (2021). YAML Ain't Markup Language (YAML™) version 1.2. [Online]. Tillgänglig på: <https://yaml.org/spec/1.2.2/>

7.3. Programvara, Verktyg och Ramverk

- Python Software Foundation. (2024). Python 3.13 Documentation. [Online]. Tillgänglig på: <https://docs.python.org/3/>
- Cisco DevNet. (2024). pyATS: Python Automated Test System. [Online]. Tillgänglig på: <https://developer.cisco.com/pyats/>
- EVE-NG. (2024). Emulated Virtual Environment Next Generation - Documentation. [Online]. Tillgänglig på: <https://www.eve-ng.net/index.php/documentation/>
- GNS3 Technologies. (2024). Graphical Network Simulator-3 Documentation. [Online]. Tillgänglig på: <https://docs.gns3.com/docs/>
- Napalm Automation. (2024). NAPALM: Network Automation and Programmability Abstraction Layer with Multi-vendor support. [Online]. Tillgänglig på: <https://napalm.readthedocs.io/>

- Nornir Team. (2024). Nornir: A type-safe multi-threaded framework for network automation. [Online]. Tillgänglig på: <https://nornir.tech/>
- Red Hat Ansible. (2024). Ansible for Network Automation. [Online]. Tillgänglig på: <https://www.ansible.com/integrations/networks/>

7.4. Vetenskapliga artiklar och Rapporter

- Kim, H. & Feamster, N. (2013). Improving network management with software defined networking. IEEE Communications Magazine.